D2 – Data & Profiling: Online Retail I + II

Course: IT135IU – Introduction to Data Science
Group members:
Dang Uyen Thu - ITDSIU25044 - Product Owner
Truong Thuan Tuan- ITDSIU25051 - Data Engineer
Nguyen An Khoi - ITDSIU25016 - Data Analyst
Nguyen Thi Khanh Linh - ITDSIU25019 - Visualisation lead
Nguyen Chi Phuc - ITDSIU25028 - Reproducibility lead
Dang Le Truc Linh - ITDSIU25018 - Ethics and privacy officer
Data engineer: Truong Thuan Tuan

Datasets:

1. Online Retail — https://archive.ics.uci.edu/dataset/352/online+retail — UCI ML Repository, donated by Daqing Chen — 541,909 rows, covers 01/12/2010–09/12/2011
2. Online Retail II — https://archive.ics.uci.edu/dataset/502/online+retail+ii — UCI ML Repository, donated by Daqing Chen — 1,067,371 rows, covers 01/12/2009–09/12/2011

Publisher: UCI Machine Learning Repository
Retrieved: 9/14/2026 **License:** Creative Commons Attribution 4.0 International (CC BY 4.0) — stated explicitly on both UCI pages

Important note on these two datasets: both come from the same UK gift retailer and their date ranges overlap almost completely — Online Retail II already contains the whole period covered by Online Retail, plus one extra earlier year. This notebook concatenates both, then explicitly deduplicates the overlapping transactions, with the decision logged below. This *is* the "merge two tables" operation required for D2 — it just merges by union + dedup rather than by a join key, which is worth explaining in your report.

In [1]:
```
import pandas as pd
import numpy as np
import os
import re

# Fixed random seed for reproducibility (required by the brief)
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)

RAW_DIR = "../data/raw"          # put the two downloaded .xlsx files here, unchang
PROCESSED_DIR = "../data/processed"
os.makedirs(PROCESSED_DIR, exist_ok=True)

# NOTE ON REPO SIZE / NO GIT LFS:
# The raw .xlsx files and the full merged working_dataset are NOT committed to git.
# Both raw files together are ~150MB+ and the merged dataset is a large single file
```

```
# committing either would force this repo onto Git LFS.
# Instead, data/raw/ and the full data/processed/working_dataset.* are listed in
# .gitignore, and this notebook regenerates them locally from the downloaded UCI
# files. Only small, git-friendly artifacts (profile table, cleaning log, a row
# sample) are committed -- see the final cell.
```

1. Load every raw file, unchanged

Online Retail II ships as one workbook with two sheets (`Year 2009-2010` , `Year 2010-2011`). Online Retail (352) is a single sheet. Nothing is modified in this step.

In [2]:
```
# Online Retail II — two sheets in one workbook
ii_2009 = pd.read_excel(f"{RAW_DIR}/online_retail_II.xlsx", sheet_name="Year 2009-2
ii_2010 = pd.read_excel(f"{RAW_DIR}/online_retail_II.xlsx", sheet_name="Year 2010-2
retail_ii_raw = pd.concat([ii_2009, ii_2010], ignore_index=True)

# Online Retail (352) — single sheet
retail_i_raw = pd.read_excel(f"{RAW_DIR}/Online Retail.xlsx")

print(f"Online Retail (352):    {retail_i_raw.shape[0]:>9,} rows x {retail_i_raw.sh
print(f"Online Retail II (502): {retail_ii_raw.shape[0]:>9,} rows x {retail_ii_raw.
print("\nOnline Retail (352) columns:  ", list(retail_i_raw.columns))
print("Online Retail II (502) columns:", list(retail_ii_raw.columns))
```

```
Online Retail (352):      541,909 rows x 8 cols
Online Retail II (502): 1,067,371 rows x 8 cols

Online Retail (352) columns:   ['InvoiceNo', 'StockCode', 'Description', 'Quantity',
'InvoiceDate', 'UnitPrice', 'CustomerID', 'Country']
Online Retail II (502) columns: ['Invoice', 'StockCode', 'Description', 'Quantity',
'InvoiceDate', 'Price', 'Customer ID', 'Country']
```

2. Profile each raw table

Rows, columns, dtypes, missing values, and ranges — goes into your D2 report (Section 2).

In [3]:
```
def profile_table(name, df):
    rows = []
    for col in df.columns:
        s = df[col]
        row = {
            "table": name,
            "column": col,
            "dtype": str(s.dtype),
            "missing_count": int(s.isna().sum()),
            "missing_pct": round(100 * s.isna().mean(), 2),
        }
        if pd.api.types.is_numeric_dtype(s):
            row["min"] = s.min()
            row["max"] = s.max()
        else:
            row["min"] = None
```

```
            row["max"] = None
        rows.append(row)
    return pd.DataFrame(rows)

data_profile = pd.concat([
    profile_table("online_retail_I", retail_i_raw),
    profile_table("online_retail_II", retail_ii_raw),
], ignore_index=True)

data_profile.to_csv(f"{PROCESSED_DIR}/data_profile.csv", index=False)
data_profile
```

Out[3]:

	table	column	dtype	missing_count	missing_pct	min	ma
0	online_retail_I	InvoiceNo	object	0	0.00	NaN	Na
1	online_retail_I	StockCode	object	0	0.00	NaN	Na
2	online_retail_I	Description	object	1454	0.27	NaN	Na
3	online_retail_I	Quantity	int64	0	0.00	-80995.00	80995
4	online_retail_I	InvoiceDate	datetime64[us]	0	0.00	NaN	Na
5	online_retail_I	UnitPrice	float64	0	0.00	-11062.06	38970
6	online_retail_I	CustomerID	float64	135080	24.93	12346.00	18287
7	online_retail_I	Country	str	0	0.00	NaN	Na
8	online_retail_II	Invoice	object	0	0.00	NaN	Na
9	online_retail_II	StockCode	object	0	0.00	NaN	Na
10	online_retail_II	Description	object	4382	0.41	NaN	Na
11	online_retail_II	Quantity	int64	0	0.00	-80995.00	80995
12	online_retail_II	InvoiceDate	datetime64[us]	0	0.00	NaN	Na
13	online_retail_II	Price	float64	0	0.00	-53594.36	38970
14	online_retail_II	Customer ID	float64	243007	22.77	12346.00	18287
15	online_retail_II	Country	str	0	0.00	NaN	Na

3. Cleaning log

Every cleaning / merging decision, in order, with a reason.

In [4]:

```
cleaning_log = []

def log_step(action, reason, rows_before, rows_after):
    cleaning_log.append({
        "action": action,
```

```
        "reason": reason,
        "rows_before": rows_before,
        "rows_after": rows_after,
    })
```

4. Align column names, then merge (union + dedup)

Online Retail II uses `Invoice`, `Price`, `Customer ID`; Online Retail (352) uses `InvoiceNo`, `UnitPrice`, `CustomerID`. Rename to a common schema before combining, then drop the rows that are duplicated because the two datasets cover overlapping dates.

In [5]:
```
retail_ii = retail_ii_raw.rename(columns={
    "Invoice": "InvoiceNo",
    "Price": "UnitPrice",
    "Customer ID": "CustomerID",
})
retail_ii["source_file"] = "online_retail_II"

retail_i = retail_i_raw.copy()
retail_i["source_file"] = "online_retail_I"

log_step(
    "rename columns on Online Retail II to match Online Retail (352) schema",
    "Invoice->InvoiceNo, Price->UnitPrice, 'Customer ID'->CustomerID so the two tab
    len(retail_ii), len(retail_ii)
)

print("Retail I date range: ", retail_i["InvoiceDate"].min(), "to", retail_i["Invoi
print("Retail II date range:", retail_ii["InvoiceDate"].min(), "to", retail_ii["Inv

common_cols = ["InvoiceNo", "StockCode", "Description", "Quantity",
               "InvoiceDate", "UnitPrice", "CustomerID", "Country"]

combined = pd.concat([retail_i, retail_ii], ignore_index=True)
before = len(combined)
combined = combined.drop_duplicates(subset=common_cols, keep="first")
after = len(combined)

log_step(
    "concat(online_retail_I, online_retail_II) then drop_duplicates on common trans
    f"Both datasets cover overlapping dates (same retailer); dropped {before - afte
    before, after
)

print(f"Combined, deduplicated dataset: {after:,} rows")
```

```
Retail I date range:  2010-12-01 08:26:00 to 2011-12-09 12:50:00
Retail II date range: 2009-12-01 07:45:00 to 2011-12-09 12:50:00
Combined, deduplicated dataset: 1,033,036 rows
```

5. Basic cleaning

Cancellations (InvoiceNo starting with 'C'), missing CustomerID, and non-positive prices/quantities are common features of this dataset. Negative quantities and prices are retained as valid transaction values because they represent reversals or corrections; cancellations are also retained as a flag. The derived `line_total` measures transaction revenue, not profit, because the dataset does not contain product cost. The source does not provide product categories, so `Category` is derived from `Description` with ordered keyword rules. This heuristic is not an official taxonomy; unmatched descriptions are labeled `Other / Unclassified`, and the first matching rule wins. Negative `Quantity` values are kept as return/reversal quantities, while cancellation invoices are tracked separately.

In [6]:
```
combined["InvoiceNo"] = combined["InvoiceNo"].astype(str)
combined["is_cancellation"] = combined["InvoiceNo"].str.startswith("C")

log_step(
    "derive is_cancellation from InvoiceNo prefix 'C'",
    "Cancelled invoices are a documented convention in this dataset; kept as a flag
    len(combined), len(combined)
)


CATEGORY_KEYWORDS = {
    "Kitchen & Dining": (
        "MUG", "MUGS", "CUP", "CUPS", "TEACUP", "TEA CUPS", "TEAPOT", "TEA POT",
        "SAUCER", "PLATE", "PLATES", "BOWL", "BOWLS", "CUTLERY", "SPOON", "FORK",
        "KNIFE", "KETTLE", "JUG", "DISH", "DISHES", "KITCHEN", "PANTRY", "APRON",
        "OVEN", "CAKE", "BAKING", "MOULD", "MOULDS", "COOKING", "CHOPPING", "TRAY",
        "JAR", "JARS", "COASTER", "COASTERS", "GLASS BOWL", "BREAKFAST SET", "POPCO
    ),
    "Home Decor": (
        "CANDLE", "CANDLES", "T-LIGHT", "TEALIGHT", "HOME DECOR", "VASE", "VASES",
        "FRAME", "FRAMES", "DOORMAT", "WREATH", "ORNAMENT", "MIRROR", "CLOCK", "COA
        "CUSHION", "CUSHIONS", "GARLAND", "CABINET", "CHEST", "CHALKBOARD", "WALL S
        "TRINKET", "HEART", "WICKER",
    ),
    "Toys & Games": (
        "TOY", "TOYS", "GAME", "GAMES", "PUZZLE", "PUZZLES", "PLAYING CARDS", "DOLL
        "DOLLY", "TEDDY", "SKIPPING ROPE", "KITE", "MARBLES", "JIGSAW", "BUILDING B
    ),
    "Stationery & Gifts": (
        "CARD", "CARDS", "NOTEBOOK", "NOTEBOOKS", "PEN", "PENCIL", "STENCIL", "CRAF
        "STICKER", "GIFT WRAP", "WRAP", "TAG", "TISSUE", "RIBBON", "ENVELOPE", "PAP
    ),
    "Bags & Accessories": (
        "BAG", "BAGS", "PURSE", "WALLET", "POUCH", "PARASOL", "UMBRELLA", "KEYRING"
        "KEY RING", "LUGGAGE", "SUITCASE",
    ),
    "Clothing & Accessories": (
        "SCARF", "HAT", "GLOVE", "SOCK", "SHIRT", "T-SHIRT", "DRESS", "SKIRT", "JAC
        "COAT", "BOOT", "SHOE", "SLIPPER", "MITTEN", "HEADBAND", "HAIRBAND", "BRACE
        "NECKLACE", "EARRING", "JEWEL", "TIE",
    ),
    "Garden & Outdoor": (
        "GARDEN", "PLANTER", "WATERING", "FLOWER", "PLANT", "KNEELING", "BIRD BATH"
```

```
        "WHEELBARROW", "SEED", "OUTDOOR",
    ),
    "Lighting": ("FAIRY LIGHT", "LIGHT", "LIGHTS", "LAMP", "LAMPS", "BULB", "BULBS"
}
description = combined["Description"].fillna("").astype(str).str.upper()
combined["Category"] = pd.NA
for category, keywords in CATEGORY_KEYWORDS.items():
    pattern = r"\b(?:" + "|".join(re.escape(keyword) for keyword in keywords) + r")
    matches = description.str.contains(pattern, regex=True, na=False)
    assign = combined["Category"].isna() & matches
    combined.loc[assign, "Category"] = category
combined["Category"] = combined["Category"].fillna("Other / Unclassified")

log_step(
    "derive Category from Description using ordered keyword rules",
    "Heuristic product grouping; first matching category wins and unmatched descrip
    len(combined), len(combined)
)


log_step(
    "retain negative quantities and prices",
    "Negative values represent transaction reversals or corrections; retain them fo
    len(combined), len(combined)
)

before = len(combined)
missing_customer = combined["CustomerID"].isna().sum()
log_step(
    "note missing CustomerID (not dropped yet)",
    f"{missing_customer:,} rows have no CustomerID; decide with your Data Analyst w
    before, before
)

combined["InvoiceDate"] = pd.to_datetime(combined["InvoiceDate"], errors="coerce")
log_step(
    "to_datetime(InvoiceDate)",
    "Needed as datetime for any time-based analysis",
    len(combined), len(combined)
)

pd.DataFrame(cleaning_log)
```

Out[6]:

	action	reason	rows_before	rows_after
0	rename columns on Online Retail II to match On...	Invoice->InvoiceNo, Price->UnitPrice, 'Custome...	1067371	1067371
1	concat(online_retail_I, online_retail_II) then...	Both datasets cover overlapping dates (same re...	1609280	1033036
2	derive is_cancellation from InvoiceNo prefix 'C'	Cancelled invoices are a documented convention...	1033036	1033036
3	derive Category from Description using ordered...	Heuristic product grouping; first matching cat...	1033036	1033036
4	retain negative quantities and prices	Negative values represent transaction reversal...	1033036	1033036
5	note missing CustomerID (not dropped yet)	235,151 rows have no CustomerID; decide with y...	1033036	1033036
6	to_datetime(InvoiceDate)	Needed as datetime for any time-based analysis	1033036	1033036

6. Required D2 operations: filter, group/aggregate, merge, derived column

The merge is in Section 4 above. These are the remaining three, with output shown.

In [7]:
```
# (a) Derived column: transaction revenue = Quantity x UnitPrice
combined["line_total"] = combined["Quantity"] * combined["UnitPrice"]

# (b) Filter: exclude cancellations for a "genuine sales" view
genuine_sales = combined[~combined["is_cancellation"]]
print("Genuine sales rows:", len(genuine_sales))

# (c) Group + aggregate: revenue by country
revenue_by_country = (
    genuine_sales.groupby("Country")["line_total"]
    .sum()
    .sort_values(ascending=False)
)
print(revenue_by_country.head(10))
```

```
Genuine sales rows: 1013932
Country
United Kingdom    1.725152e+07
EIRE              6.587673e+05
Netherlands       5.540381e+05
Germany           4.250197e+05
France            3.504561e+05
Australia         1.692835e+05
Spain             1.083325e+05
Switzerland       1.006856e+05
Sweden            9.186982e+04
Denmark           6.858069e+04
Name: line_total, dtype: float64
```

7. Save the working dataset

In [8]:
```
# Save the working dataset LOCALLY only (this file is .gitignore'd -- see repo root
# Saved as CSV so no extra dependencies (pyarrow etc.) are needed.
combined.to_csv(f"{PROCESSED_DIR}/working_dataset.csv", index=False)

# Small, git-friendly artifacts that DO get committed:
# 1) The data profile table (already saved above, a few KB)
# 2) The cleaning log (already saved above, a few KB)
# 3) A small random sample of the working dataset, so a reader can open the repo
#    on GitHub and see the real column layout without downloading anything.
sample = combined.sample(n=min(2000, len(combined)), random_state=RANDOM_SEED)
sample.to_csv(f"{PROCESSED_DIR}/working_dataset_sample.csv", index=False)

pd.DataFrame(cleaning_log).to_csv(f"{PROCESSED_DIR}/cleaning_log.csv", index=False)

print(f"Saved working_dataset.csv locally with {len(combined):,} rows and {combined
print(f"Committed to repo instead: data_profile.csv, cleaning_log.csv, "
      f"working_dataset_sample.csv ({len(sample):,} rows)")
print("Full dataset is reproducible by re-running this notebook against the raw "
      "files listed in the README -- no Git LFS needed.")
```

```
Saved working_dataset.csv locally with 1,033,036 rows and 12 columns
Committed to repo instead: data_profile.csv, cleaning_log.csv, working_dataset_sampl
e.csv (2,000 rows)
Full dataset is reproducible by re-running this notebook against the raw files liste
d in the README -- no Git LFS needed.
```

8. Why no Git LFS

This project keeps the git repository small on purpose, so nothing needs Git LFS:

- **Raw files** (`data/raw/*.xlsx` , ~150MB+ combined) are downloaded straight from UCI by each collaborator and are **not committed** -- see the README for the two download links and the exact filenames the notebook expects.
- **The full merged working dataset** (`data/processed/working_dataset.csv` , ~1M+ rows) is also **not committed**. It is fully reproducible: re-running this notebook end-to-end regenerates it from the raw files, deterministically (fixed `RANDOM_SEED`).

- **What *is* committed** to `data/processed/`: `data_profile.csv`, `cleaning_log.csv`, and `working_dataset_sample.csv` (a 2,000-row sample) -- all a few KB to a few MB, so plain git handles them with no LFS filter needed.
- `.gitignore` in the repo root excludes `data/raw/` and `data/processed/working_dataset.csv` so nobody accidentally commits the large files later.